

FundWave — Product Requirements Document (PRD)

Version: 1.0

Author: Valli (dnvpravallika)

Date: July 2026

Status: In Progress

1. Overview

1.1 Product Summary

FundWave is a full-stack personal finance tracker that helps users track income, expenses, savings goals, and get AI-powered financial insights — all in one place. Built with the MERN stack and TypeScript, containerized with Docker, and deployed with a modern DevOps pipeline.

1.2 Problem Statement

Most people struggle to track where their money goes. Generic spreadsheets are tedious, and most finance apps are either too complex or paid. FundWave solves this with a clean, smart, and free personal finance tracker with AI insights that actually help users understand and improve their financial habits.

1.3 Target Users

- Students and young professionals managing personal finances
- Individuals who want a simple dashboard without complex tools
- Anyone wanting AI-powered insights on their spending patterns

1.4 Goals

- Give users a real-time view of their financial health
 - Help users make smarter financial decisions using AI insights
 - Serve as a strong portfolio project showcasing full-stack + DevOps skills
-

2. Tech Stack

Layer	Technology
Frontend	React, TypeScript, TailwindCSS, Recharts
Backend	Node.js, Express.js, TypeScript
Database	MongoDB Atlas (Free Tier)
Authentication	JWT + Bcrypt
AI Insights	Google Gemini API (Free Tier)
Hosting — Frontend	Vercel (Free)
Hosting — Backend	Render (Free)
Containerization	Docker + Docker Compose
Orchestration	Kubernetes (Minikube — local)
CI/CD	GitHub Actions
Keep-Alive	cron-job.org (pings Render every 5 min)

3. Features

3.1 Phase 1 — Core (MVP)

3.1.1 Authentication

- User registration with name, email, password
- Login with JWT token
- Protected routes — only logged-in users can access dashboard
- Logout functionality
- Password hashing with Bcrypt

3.1.2 Dashboard — At a Glance

- Total Balance card
- Total Income card
- Total Expenses card
- Total Savings card
- All updated in real time as transactions are added

3.1.3 Transaction Management

- Add transaction (income or expense)
- Fields: title, amount, category, date, type (income/expense), notes (optional)
- Edit transaction
- Delete transaction
- View all transactions in a list (sorted by latest date)

3.1.4 Categories

- Predefined categories: Food, Rent, Travel, Shopping, Entertainment, Health, Salary, Freelance, Investment, Other
- Color coded per category

3.1.5 Filters

- Filter transactions by date range
 - Filter by category
 - Filter by type (income/expense)
-

3.2 Phase 2 — Enhanced Features

3.2.1 Charts and Visualizations

- Pie chart — expense breakdown by category
- Bar chart — monthly income vs expenses comparison
- Line chart — balance trend over last 6 months
- All charts built with Recharts

3.2.2 Budget Management

- Set monthly budget per category (e.g. Food: ₹5000/month)
- Visual progress bar showing how much of budget is used
- Color changes: green → yellow → red as budget fills up
- Alert when budget is 80% used
- Alert when budget is exceeded

3.2.3 Recurring Transactions

- Mark a transaction as recurring (weekly/monthly)
- Auto-add recurring transactions on due date
- Examples: rent, subscriptions, salary

3.2.4 Export

- Export transactions to CSV

- Export transactions to PDF
- Filter before export (date range, category)

3.2.5 Dark/Light Mode

- Toggle between dark and light theme
 - Preference saved in localStorage
-

3.3 Phase 3 — Standout Features

3.3.1 AI Insights (Gemini API)

- Monthly AI-generated financial summary
- Examples of insights:
 - "You spent 40% more on food this month compared to last month"
 - "You are on track to hit your savings goal this month"
 - "Your top expense category is Entertainment — consider setting a budget"
 - "You have saved ₹2000 more than last month. Great job!"
- Triggered on demand (user clicks "Get Insights" button)
- Powered by Google Gemini API (free tier)

3.3.2 Smart Categorization

- When user types a transaction title, AI suggests the category automatically
- Example: type "Swiggy" → auto-suggests "Food"
- Example: type "Netflix" → auto-suggests "Entertainment"
- User can accept or override the suggestion

3.3.3 Savings Goals

- Create a savings goal with a target amount and deadline
- Example: "New Laptop — ₹50,000 by December 2026"
- Visual progress bar showing current savings vs target
- AI tip on how to reach the goal faster

3.3.4 Multi-Currency Support

- Select preferred currency (INR, USD, EUR, GBP etc.)
- All amounts displayed in selected currency
- Currency preference saved per user profile

3.3.5 Email Alerts (Optional/Stretch)

- Email notification when budget exceeded
- Weekly financial summary email

- Powered by NodeMailer (free)

4. System Architecture

```
User Browser
  ↓
React Frontend (Vercel)
  ↓ REST API calls
Express Backend (Render)
  ↓
MongoDB Atlas (Database)
  ↓
Gemini API (AI Insights)
```

DevOps Architecture (Resume Showcase)

```
GitHub Repository
  ↓ push to main
GitHub Actions (CI/CD)
  ↓ build & test
Docker Image built
  ↓
Docker Hub (image pushed)
  ↓
Kubernetes Manifests (Minikube local)
├─ frontend-deployment.yaml
├─ backend-deployment.yaml
├─ service.yaml
└─ ingress.yaml
```

5. API Design

Auth Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login user, return JWT
GET	/api/auth/me	Get logged in user profile

Transaction Endpoints

Method	Endpoint	Description
GET	/api/transactions	Get all transactions (with filters)
POST	/api/transactions	Add new transaction
PUT	/api/transactions/:id	Update transaction
DELETE	/api/transactions/:id	Delete transaction

Budget Endpoints

Method	Endpoint	Description
GET	/api/budgets	Get all budgets
POST	/api/budgets	Set budget for a category
PUT	/api/budgets/:id	Update budget

Savings Goals Endpoints

Method	Endpoint	Description
GET	/api/goals	Get all savings goals
POST	/api/goals	Create new goal
PUT	/api/goals/:id	Update goal progress
DELETE	/api/goals/:id	Delete goal

AI Endpoints

Method	Endpoint	Description
POST	/api/ai/insights	Generate AI financial insights
POST	/api/ai/categorize	Smart categorize a transaction

6. Data Models

User

typescript

```
{
  _id: ObjectId,
  name: string,
  email: string (unique),
  password: string (hashed),
  currency: string (default: "INR"),
  createdAt: Date
}
```

Transaction

typescript

```
{
  _id: ObjectId,
  userId: ObjectId (ref: User),
  title: string,
  amount: number,
  type: "income" | "expense",
  category: string,
  date: Date,
  notes: string (optional),
  isRecurring: boolean,
  recurringFrequency: "weekly" | "monthly" | null,
  createdAt: Date
}
```

Budget

typescript

```
{
  _id: ObjectId,
  userId: ObjectId (ref: User),
  category: string,
  limit: number,
  month: string (e.g. "2026-07"),
  createdAt: Date
}
```

Savings Goal

```
typescript

{
  _id: ObjectId,
  userId: ObjectId (ref: User),
  title: string,
  targetAmount: number,
  currentAmount: number,
  deadline: Date,
  createdAt: Date
}
```

7. UI Pages

Page	Description
Login / Register	Auth pages (already built)
Dashboard	Balance, income, expense, savings cards + charts
Transactions	List of all transactions + add/edit/delete
Budget	Set budgets per category, progress bars
Savings Goals	Create and track goals
AI Insights	Click to generate Gemini AI insights
Settings	Currency preference, profile, logout

8. DevOps Plan

8.1 Docker

- Dockerfile for frontend (React build served via Nginx)
- Dockerfile for backend (Node.js)
- docker-compose.yml to run everything locally together

8.2 Kubernetes (Local — Minikube)

- frontend-deployment.yaml
- backend-deployment.yaml
- mongo-deployment.yaml (local only)

- service.yaml for each
- ingress.yaml for routing
- ConfigMap for environment variables

8.3 CI/CD — GitHub Actions

- On push to main branch:
 - Run tests
 - Build Docker images
 - Push to Docker Hub
 - Deploy to Render (backend) via webhook
 - Deploy to Vercel (frontend) automatically

8.4 Keep Render Alive

- Use cron-job.org
- Ping backend health endpoint every 5 minutes
- Prevents Render free tier from sleeping

9. Project Phases & Timeline

Phase	Features	Timeline
Phase 1	Auth + Dashboard + Transactions	Week 1-2
Phase 2	Charts + Budget + Filters + Export	Week 3-4
Phase 3	AI Insights + Goals + Multi-currency	Week 5-6
DevOps	Docker + K8s + CI/CD	Week 7
Polish	UI polish + README + Documentation	Week 8

10. Resume Highlights (What to mention)

- Built a full-stack personal finance tracker using MERN + TypeScript
- Integrated Google Gemini AI API for smart financial insights and auto-categorization
- Implemented JWT authentication, budget tracking, savings goals, and CSV/PDF export
- Containerized application using Docker and Docker Compose
- Wrote Kubernetes manifests and deployed locally on Minikube
- Built CI/CD pipeline using GitHub Actions — auto deploys on push to main

- Deployed frontend on Vercel and backend on Render with zero-cost infrastructure
 - Designed RESTful APIs with Express.js and MongoDB Atlas
-

11. GitHub Repository Structure

```
fundwave/
├── client/                                # React Frontend
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── context/
│   │   └── utils/
│   ├── Dockerfile
│   └── package.json
├── server/                               # Express Backend
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── middleware/
│   │   └── utils/
│   ├── Dockerfile
│   └── package.json
├── k8s/                                  # Kubernetes Manifests
│   ├── frontend-deployment.yaml
│   ├── backend-deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
├── .github/
│   └── workflows/
│       └── ci-cd.yml                    # GitHub Actions
├── docker-compose.yml
└── README.md
```